SOFTWARE MAINTENANCE

Software Maintenance

Software maintenance refers to the process of modifying and updating a software system after it has been delivered to the customer. This can include fixing bugs, adding new features, improving performance, or updating the software to work with new hardware or software systems. The goal of software maintenance is to keep the software system working correctly, efficiently, and securely, and to ensure that it continues to meet the needs of the users.

There are several key aspects of software maintenance, including:

- Bug fixing:

The process of finding and fixing errors and problems in the software.

- Enhancements:

The process of adding new features or improving existing features to meet the evolving needs of the users.

- Performance optimization:

The process of improving the speed, efficiency, and reliability of the software.

- Porting and migration:

The process of adapting the software to run on new hardware or software platforms.

- Re-engineering:

The process of improving the design and architecture of the software to make it more maintainable and scalable.

- Documentation:

The process of creating, updating, and maintaining the documentation for the software, including user manuals, technical specifications, and design documents.

Types of maintenance

In a software lifetime, type of maintenance may vary based on its nature. It may be just a routine maintenance tasks as some bug discovered by some user or it may be a large event in itself based on maintenance size or nature. Following are some types of maintenance based on their characteristics:

- Corrective Maintenance –

This includes modifications and updations done in order to correct or fix problems, which are either discovered by user or concluded by user error reports.

- Adaptive Maintenance –

This includes modifications and updations applied to keep the software product up-to date and tuned to the ever changing world of technology and business environment.

- Perfective Maintenance –

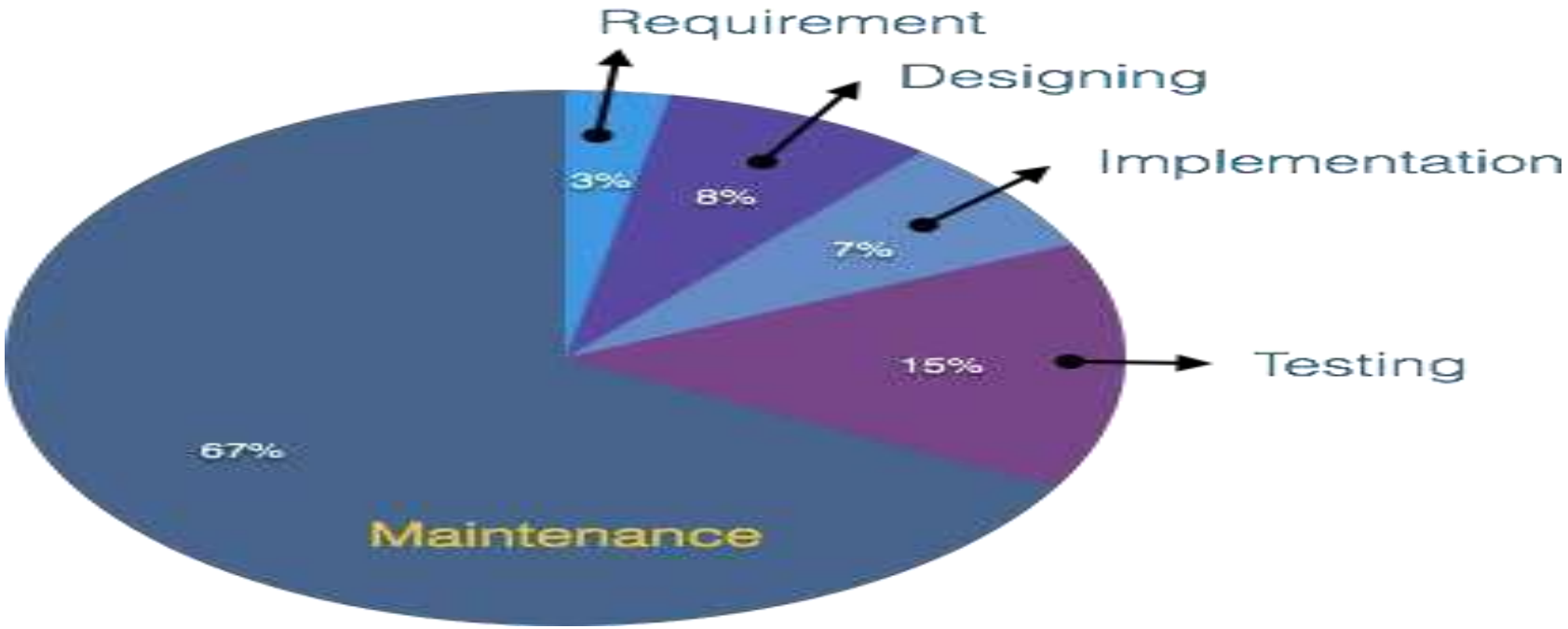
This includes modifications and updates done in order to keep the software usable over long period of time. It includes new features, new user requirements for refining the software and improve its reliability and performance.

- Preventive Maintenance –

This includes modifications and updations to prevent future problems of the software. It aims to attend problems, which are not significant at this moment but may cause serious issues in future.

Cost of Maintenance

Reports suggest that the cost of maintenance is high. A study on estimating software maintenance found that the cost of maintenance is as high as 67% of the cost of entire software process cycle.



On an average, the cost of software maintenance is more than 50% of all SDLC phases.

There are various factors, which trigger maintenance cost go high, such as:

Real-world factors affecting Maintenance Cost

- The standard age of any software is considered up to 10 to 15 years.
- Older softwares, which were meant to work on slow machines with less memory and storage capacity cannot keep themselves challenging against newly coming enhanced softwares on modern hardware.
- As technology advances, it becomes costly to maintain old software.
- Most maintenance engineers are newbie and use trial and error method to rectify problem.
- Often, changes made can easily hurt the original structure of the software, making it hard for any subsequent changes.
- Changes are often left undocumented which may cause more conflicts in future.

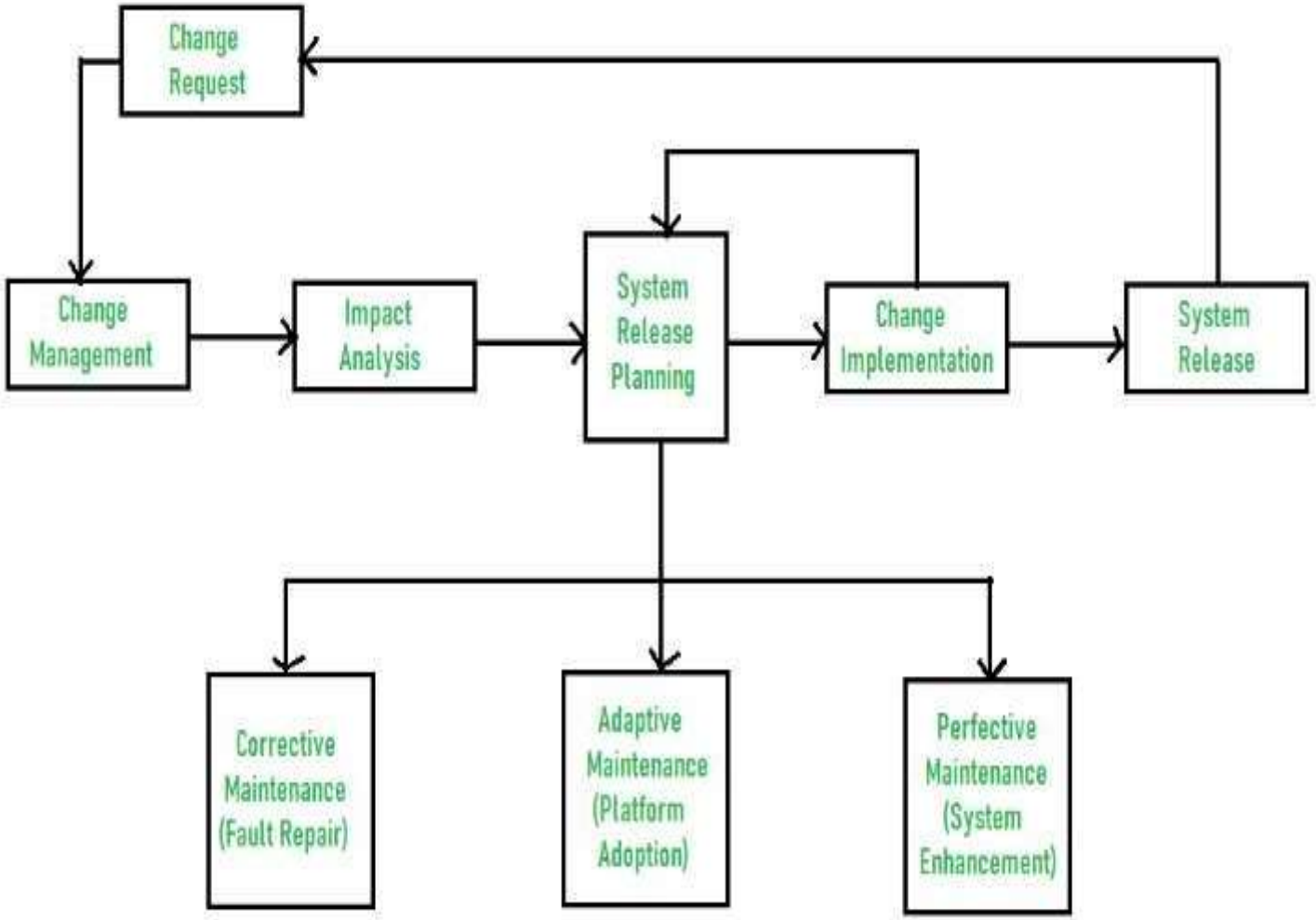
Software-end factors affecting Maintenance Cost

- Structure of Software Program
- Programming Language
- Dependence on external environment
- Staff reliability and availability

Need for Maintenance

- Software Maintenance must be performed in order to:
- Correct faults.
- Improve the design.
- Implement enhancements.
- Interface with other systems.
- Accommodate programs so that different hardware, software, system features, and telecommunications facilities can be used.
- Migrate legacy software.
- Retire software.
- Requirement of user changes.
- Run the code fast

Components of Software Maintenance Process



MAINTENANCE PROCESS

1. Change Request

In the maintenance process initially, the request for change is made.

2. Change Management

In this place, the status of all the change requests is identified and described.

3. Impact Analysis

Following activities are performed in this place:

- Identify all the systems and the system products affected by a change request.
- Make an estimate of the resources needed to effect the change.
- Analyze the benefits of the change.

4. System Release Planning

In this phase, the schedule and the contents of the software release are planned. The changes can be made to all types of software maintenance.

5. Change Implementation

The implement of changes can be by first designing the changes, then coding for these changes, and finally testing the changes. Preferably the regression testing must be performed while testing the changes.

6. **System Release**

During software release, following points should be described:

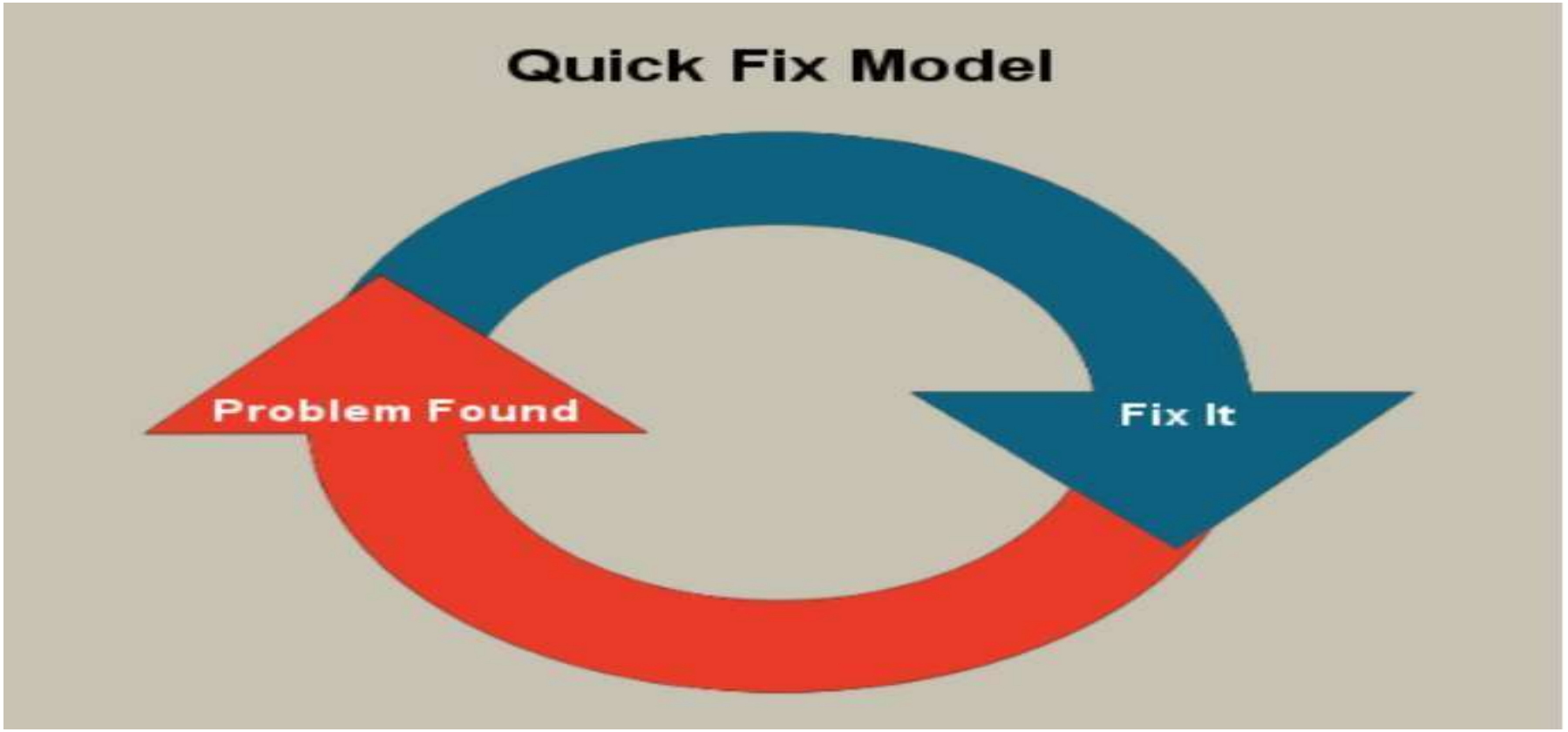
- Documentation
- Software
- Training
- Hardware Changes
- Data Conversion

Software maintenance process models

To overcome internal as well as external problems of the software, Software maintenance models are proposed. These models use different approaches and techniques to simplify the process of maintenance as well as to make is cost effective. Software maintenance models that are of most importance are:

- **Quick-Fix Model**

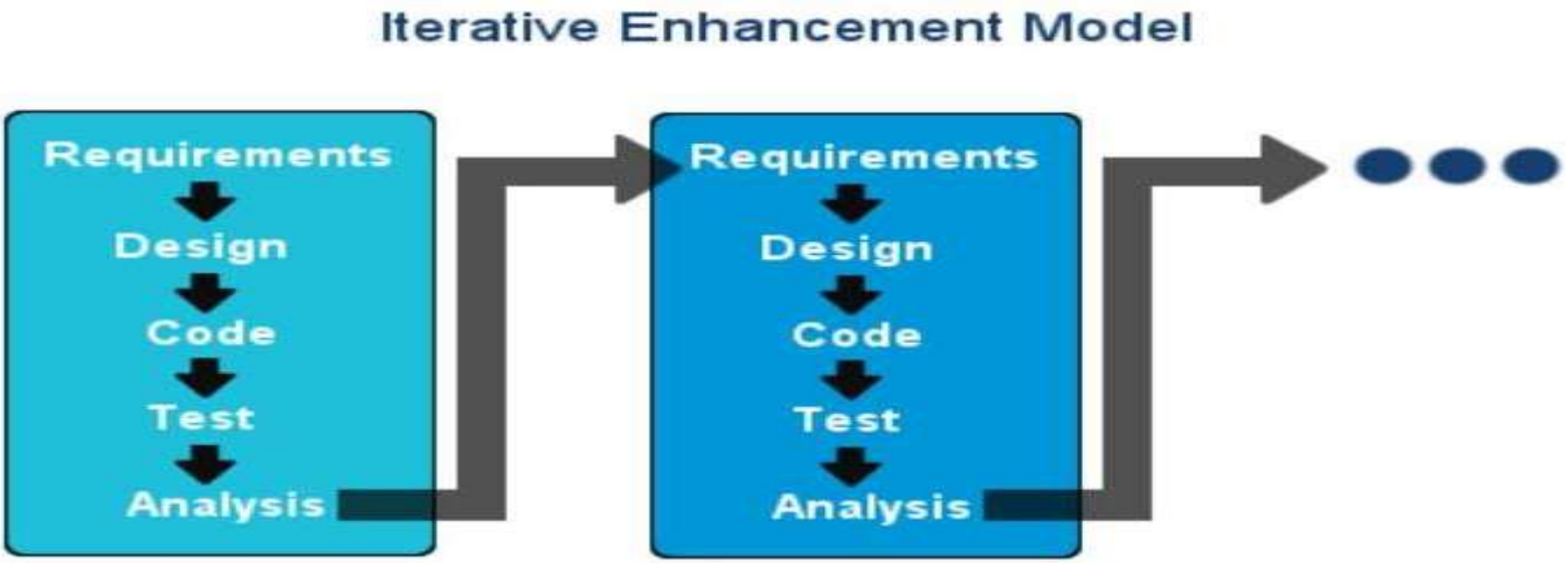
This is an ad hoc approach used for maintaining the software system. The objective of this model is to identify the problem and then fix it as quickly as possible. The advantage is that it performs its work quickly and at a low cost. This model is an approach to modify the software code with little consideration for its impact on the overall structure of the software system.



- **Iterative Enhancement Model**

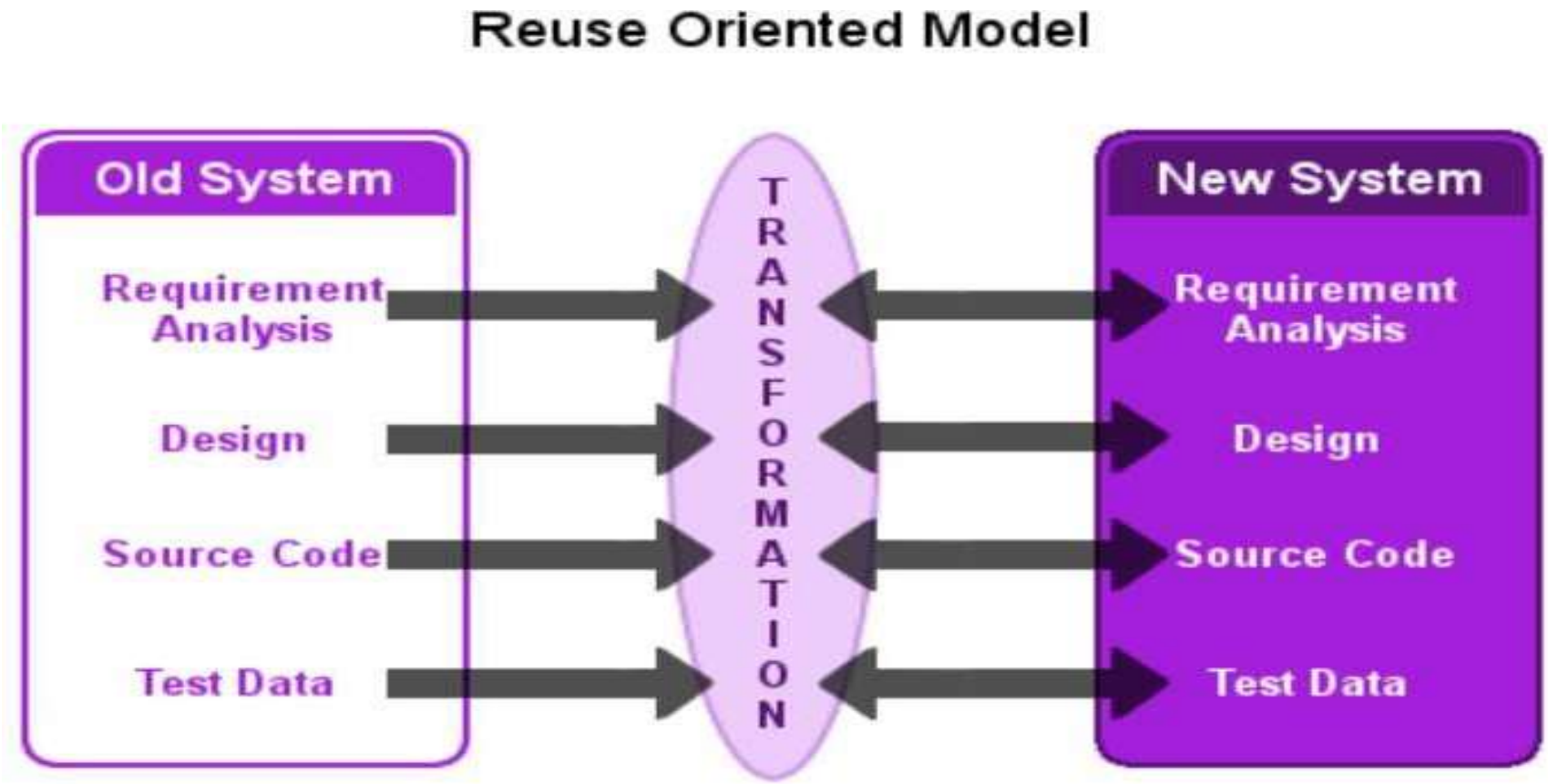
Iterative enhancement model considers the changes made to the system are iterative in nature. This model incorporates changes in the software based on the analysis of the existing system. It assumes complete documentation of the software is available in the beginning. Moreover, it attempts to control complexity and tries to maintain good design. Iterative Enhancement Model is divided into three stages:

- Analysis of software system.
- Classification of requested modifications.
- Implementation of requested modifications.



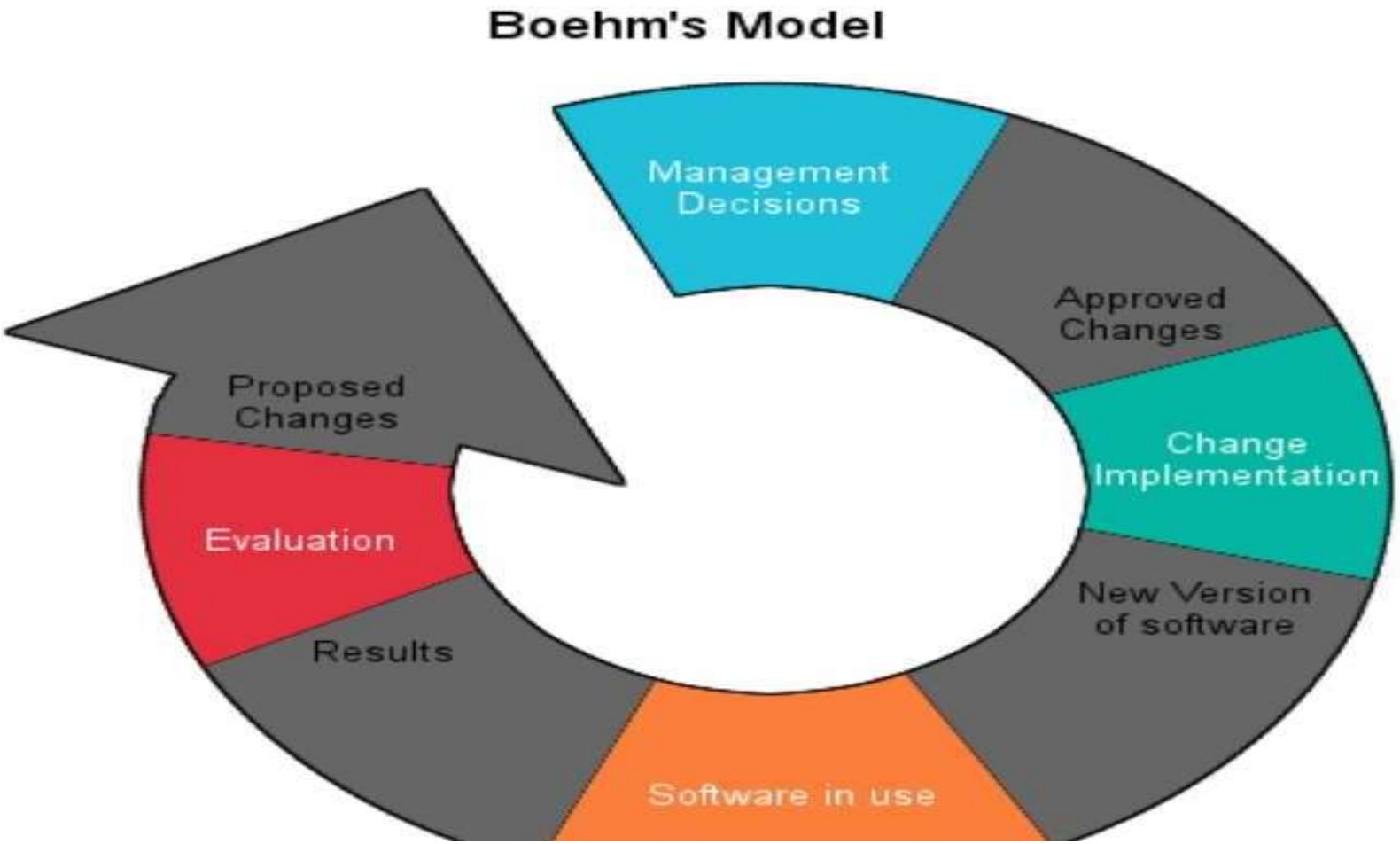
- **The Re-use Oriented Model**

The parts of the old/existing system that are appropriate for reuse are identified and understood, in Reuse Oriented Model. These parts are then go through modification and enhancement, which are done on the basis of the specified new requirements. The final step of this model is the integration of modified parts into the new system.



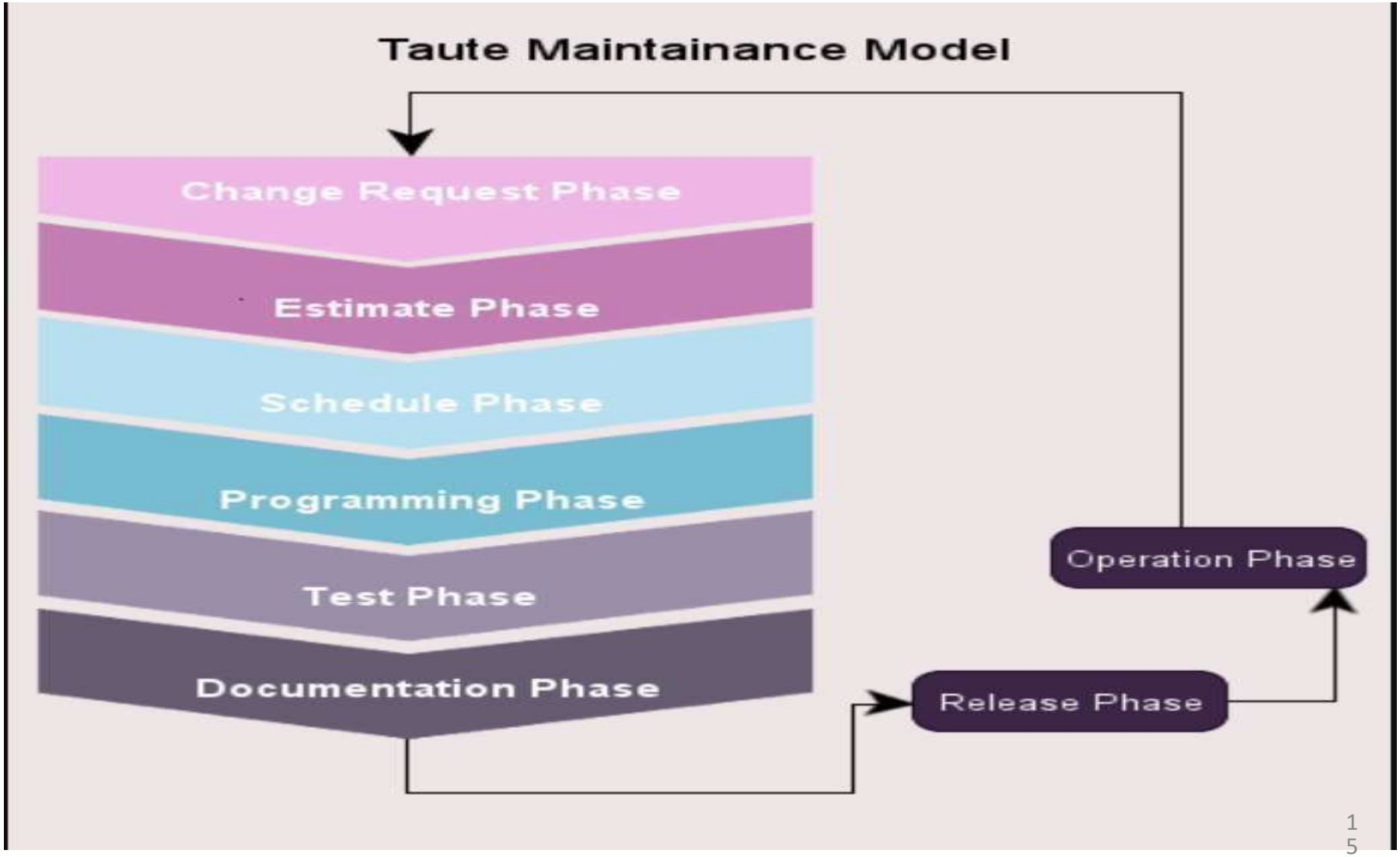
- **Boehm's Model**

Boehm’s Model performs maintenance process based on the economic models and principles. It represents the maintenance process in a closed loop cycle, wherein changes are suggested and approved first and then are executed.



- **Taute Maintenance Model:**

Taute's model is a typical maintenance model that consists of eight phases in cycle fashion. The process of maintenance begins by requesting the change and ends with its operation. The phases of Taute's Maintenance Model are:



Maintenance metrics

- **Mean time to repair (MTTR)**

MTTR is the measurement of the average time taken to fix a piece of equipment or machinery after it has malfunctioned.

MTTR= total repair time / total number of repairs

- **Emergency work orders (EWOs)**

EWO is an unplanned or emergency maintenance task that requires immediate attention to prevent further damage or safety hazards. EWOs are a critical measure of maintenance team responsiveness and agility. It is calculated by,

EWOs = Total emergency work orders / Total number of work orders

- **Planned maintenance percentage (PMP)**

Planned maintenance percentage is a measure of the percentage of maintenance tasks that are planned and scheduled in advance. It's like a forecast for maintenance work that needs to be carried out in the future, ensuring that everything is in order before it becomes a problem.

PMP = Planned maintenance hours / Total maintenance hours × 100

- **Mean time between failures (MTBF)**

Mean time between failures (MTBF) is a measure of how often equipment fails within a specific time frame or how frequently a particular type of failure is anticipated to occur.

MTBF=total number of operational hours / total number of failures

- **Overall equipment effectiveness (OEE)**

Overall equipment effectiveness (OEE) is a powerful metric that measures the performance, availability, and quality of equipment. This is like a report card for equipment, providing a comprehensive overview of how well the equipment is functioning and how it can be optimized for maximum efficiency. OEE is expressed as a percentage and calculated as:

OEE = Availability x Performance x Quality

- **Maintenance Backlogs**

Maintenance backlogs refer to the accumulation of maintenance tasks that have not been completed, either because of a lack of resources or because of other priorities. A high maintenance backlog suggests that maintenance teams are struggling to keep up with demand, leading to equipment downtime and increased maintenance costs.

Maintenance Backlogs = Number of pending work orders / Total number of work order

- **Preventive maintenance compliance (PMC)**

The percentage of tasks completed on time as per the schedule is known as PM compliance. It's an additional metric for assessing the consistency and resource-saving effectiveness of a Preventative Maintenance (PM) program. It is expressed by : **PMC = (Number of PM tasks completed on schedule / Total number of PM tasks) x 100%**

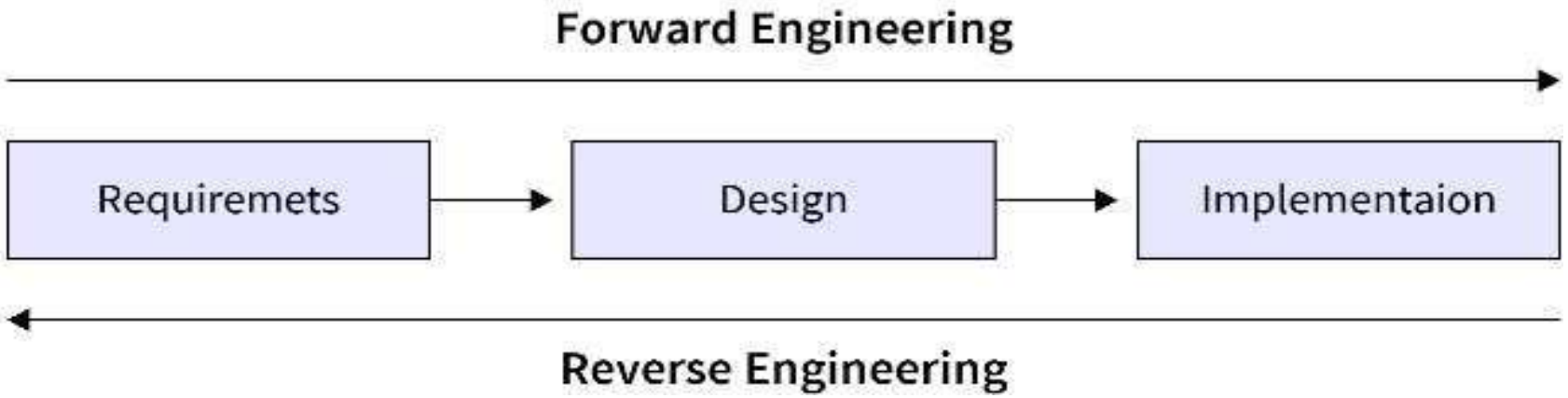
- **Equipment downtime**

Equipment downtime refers to the time that equipment is not functioning due to maintenance or unplanned issues. It can have a significant impact on business operations, causing delays, reduced productivity, and increased costs. Equipment downtime can be calculated by,

Equipment downtime =Total downtime / Total available production time x100%

Reverse Engineering

Reverse engineering in software engineering involves dissecting and analyzing existing software to understand its functionality, structure, and design. It aids in comprehending legacy systems, identifying vulnerabilities, and creating compatible alternatives. This process often involves **decompilation, disassembly, and code analysis**, allowing developers to extract valuable insights and recreate source code. Reverse engineering plays a pivotal role in **software maintenance, system migration, and security auditing**. However, its ethical and legal implications, particularly in terms of intellectual property, should be carefully considered and respected during the exploration of proprietary software.



Objective of Reverse Engineering:

- **Reducing Costs:**

Reverse engineering can help cut costs in product development by finding replacements or cost-effective alternatives for systems or components.

- **Analysis of Security:**

Reverse engineering is used in cybersecurity to examine exploits, vulnerabilities, and malware. This helps in understanding of threat mechanisms and the development of practical defenses by security experts.

- **Integration and Customization:**

Through the process of reverse engineering, developers can incorporate or modify hardware or software components into pre-existing systems to improve their operation or tailor them to meet particular needs.

- **Recovering Lost Source Code:**

Reverse engineering can be used to recover the source code of a software application that has been lost or is inaccessible or at the very least, to produce a higher-level representation of it.

- **Fixing bugs and maintenance:**

Reverse engineering can help find and repair flaws or provide updates for systems for which the original source code is either unavailable or inadequately documented.

Steps of Software Reverse Engineering

- **Initial Analysis:**

Understand the software's purpose, platform, and structure.

- **Decompile/Disassemble:**

Convert executable code back into a more human-readable form.

- **Code Analysis:**

Study the decompiled code to comprehend functionality and algorithms.

- **Identify Components:**

Isolate modules, functions, and data structures.

- **Control Flow Analysis:**

Trace program execution paths to discern logic and behavior.

- **Data Flow Analysis:**

Track data manipulation to understand input/output relationships.

- **Reconstruction:**

Rebuild higher-level abstractions for better comprehension.

- **Dynamic Analysis:**

Execute the software in a controlled environment to observe runtime behaviour.

- **API and Library Identification:**

Determine external libraries and APIs used.

- **Vulnerability Discovery:**

Identify security weaknesses, bugs, or exploits.

- **Documentation:**

Create documentation describing the software's inner workings.

- **Modification/Enhancement:**

Make changes to the software, if necessary.

- **Testing:**

Verify modified software's functionality and stability.

- **Legal and Ethical Considerations:**

Ensure compliance with software licensing and ethical guidelines.

Advantages of Reverse Engineering

- **Understanding and Documentation:**

Reverse engineering allows for a deeper understanding of how a product, software, or system works, which can aid in creating comprehensive documentation and improving future designs.

- **Innovation and Improvement:**

By dissecting existing products, engineers can identify areas for improvement and innovation, leading to the development of enhanced versions or entirely new products.

- **Interoperability:**

Reverse engineering can help decipher proprietary file formats and protocols, enabling better integration and compatibility with other systems and software.

- **Legacy System Maintenance:**

When original design documentation is unavailable, reverse engineering becomes essential for maintaining and updating legacy systems, extending their lifespan.

- **Competitive Analysis:**

Reverse engineering competitors' products can provide insights into their strengths and weaknesses, guiding strategic decisions and market positioning.

- **Troubleshooting and Debugging:**

Reverse engineering aids in diagnosing and fixing issues, especially when dealing with complex systems where the original source code or design is not accessible.

Challenges in Reverse Engineering

- **Outdated Programming Languages:**
  - Lack of modern support and tools.
  - Difficulty in finding skilled developers.
- **Lack of Documentation:**
  - Inadequate or outdated documentation.
  - Unclear system functionality and design.
- **System Dependencies and Integration:**
  - Complex interconnections with other systems.
  - Disruptions during analysis and changes.